



UNIVERSIDAD
TECNOLÓGICA
METROPOLITANA
del Estado de Chile

INFB6074 - INFRAESTRUCTURA PARA C. DE DATOS
INGENIERÍA CIVIL EN CIENCIA DE DATOS

ACTIVIDAD SEMANA 3: *Paralelismo Local y*
Procesamiento Distribuido en Data Science

Dr. Ing. Michael Miranda Sandoval

22 de abril de 2026

ACTIVIDAD SEMANA 3:

Integrantes (individual o parejas, máximo 2 integrantes):

Nombre:

Email:

Nombre:

Email:

- **Inicio formal de la actividad:** Semana 3 - Clases 1 y 2.
- **Modalidad:** Individual o parejas (máximo 2 integrantes).
- **Duración estimada:** 1 semana.

1. Instrucciones

Esta actividad está diseñada para que el estudiante analice, implemente y compare estrategias de procesamiento paralelo y procesamiento distribuido aplicadas a problemas de Data Science. El trabajo integra una fase de fundamentación teórica con una fase experimental en Python de complejidad media-avanzada, orientada a medir speedup, overhead, eficiencia, escalabilidad y costo de coordinación en distintos escenarios computacionales.

Indicaciones generales:

- La actividad puede ser realizada de forma individual o en parejas (máximo 2 integrantes).
- Siga las tres partes de la actividad: investigación teórica, aplicación práctica en Python y síntesis del informe final.
- Documente el entorno experimental: CPU, número de núcleos, memoria RAM, sistema operativo, versiones de Python y librerías, y si dispone o no de GPU.
- Utilice al menos un mecanismo de paralelismo local (`multiprocessing`, `concurrent.futures` o `joblib`) y al menos un framework de procesamiento distribuido (`Dask` o `PySpark` en modo local).
- Si no dispone de GPU, debe explicitarlo y reemplazar cualquier comparación por un análisis fundamentado entre vectorización, multicore y procesamiento distribuido local.
- Use código original; puede apoyarse en librerías estándar y documentación oficial, pero el diseño experimental y la implementación deben ser propios.
- Cite correctamente todas las fuentes consultadas y declare el uso de herramientas de apoyo como IA, notebooks de referencia o repositorios externos.
- Entregue un informe claro y bien organizado, código funcional, resultados reproducibles y visualizaciones integradas.

Lee atentamente cada sección y organiza tu trabajo para cubrir los objetivos de aprendizaje de la semana: paralelismo a nivel de datos, tareas y hardware; CPU multicore, vectorización y GPU; diferencias entre paralelismo y distribución; nodos, cluster y particionamiento; tolerancia a fallas; escalamiento vertical y horizontal; y límites reales del speedup por overhead y coordinación.

2. Etapas de la Actividad

1. **Parte 1 - Investigación teórica:** Elaborar un documento técnico que explique y compare procesamiento paralelo y procesamiento distribuido, incluyendo conceptos, ventajas, límites, errores frecuentes y criterios de decisión técnica.

2. **Parte 2 - Aplicación práctica en Python:** Implementar los experimentos solicitados, registrar métricas cuantitativas, generar visualizaciones y contrastar el comportamiento secuencial, paralelo y distribuido.
3. **Parte 3 - Informe y análisis final:** Sintetizar los hallazgos en un informe técnico con resultados, interpretación, discusión de trade-offs y recomendaciones para distintos escenarios de infraestructura.

Detalles de la parte práctica:

- 2.1. **Configuración del entorno experimental:** Documentar el sistema con Python, NumPy, Pandas, Matplotlib/Seaborn, psutil y las librerías elegidas para paralelismo y distribución. Registrar número de cores, RAM, almacenamiento, presencia de GPU y configuración del framework distribuido.
- 2.2. **Experimento A - Paralelismo a nivel de datos y vectorización:** Comparar una implementación secuencial en Python puro, una versión vectorizada con NumPy y una versión paralela local para una carga numérica intensiva sobre arreglos o matrices grandes. Medir tiempo total, speedup, eficiencia y uso de CPU.
- 2.3. **Experimento B - Paralelismo a nivel de tareas:** Construir un flujo con lotes independientes de feature engineering o inferencia batch, y comparar ejecución secuencial, `ThreadPoolExecutor` y `ProcessPoolExecutor`. Analizar el efecto del tamaño de lote, cantidad de workers y naturaleza CPU-bound o I/O-bound de la carga.
- 2.4. **Experimento C - Procesamiento distribuido sobre datos particionados:** Implementar una carga de trabajo con `Dask` o `PySpark` que incluya lectura de datos, transformaciones, al menos una agregación y una operación de combinación o `groupby`. Comparar contra una versión monolítica en Pandas y discutir cuándo la distribución aporta o no aporta valor.
- 2.5. **Experimento D - Overhead de coordinación y decisión técnica:** Modelar en Python un escenario con costos de serialización, comunicación o particionamiento, y evaluar por qué el speedup no crece linealmente. A partir de los resultados, justificar una recomendación para tres casos: servidor único, organización con múltiples fuentes de datos y plataforma de datos en cloud.

Requisitos adicionales:

- Repetir cada experimento al menos 3 veces y reportar promedio, desviación estándar y observaciones relevantes.
- Evaluar al menos 5 tamaños de entrada distintos en el Experimento A.
- Probar en el Experimento B al menos 3 configuraciones de workers y 3 tamaños de lote o chunk.
- Utilizar en el Experimento C al menos 3 configuraciones de particionamiento y una carga de trabajo con datos de tamaño suficiente para evidenciar overhead.
- Incluir al menos un caso donde el paralelismo local mejore el rendimiento y al menos un caso donde no lo mejore o incluso lo empeore.
- Incluir al menos un caso donde el procesamiento distribuido resulte justificable y otro donde la solución en un solo nodo sea preferible.
- Presentar un análisis explícito del error frecuente de asumir speedup lineal y sustentar la explicación con datos experimentales.
- Incorporar un mínimo de 4 visualizaciones: tiempo de ejecución, speedup, eficiencia o uso de recursos, y comparación entre enfoques.

* Ejemplos referenciales por experimento

Los siguientes ejemplos son solo orientativos. Su función es ayudar a delimitar el tipo de problema esperado, pero cada grupo debe definir su propio diseño experimental, tamaños de entrada, parámetros y forma de análisis.

- **Ejemplo para el Experimento A:** Comparar el cálculo de una transformación numérica intensiva, por ejemplo aplicar una combinación de operaciones como normalización, potencia, raíz y suma acumulada sobre arreglos grandes. El contraste puede realizarse entre una versión con ciclos en Python, una versión vectorizada con NumPy y una versión paralela que procese bloques independientes con `multiprocessing`.
- **Ejemplo para el Experimento B:** Dividir un conjunto de lotes independientes de datos tabulares y aplicarles una misma rutina de feature engineering, por ejemplo creación de variables agregadas, codificación básica y cálculo de estadísticas por lote. Luego comparar la ejecución secuencial con `ThreadPoolExecutor` y `ProcessPoolExecutor`, variando número de workers y tamaño de chunk.
- **Ejemplo para el Experimento C:** Procesar un conjunto de archivos CSV o Parquet particionados por fecha o por fuente de datos, aplicar limpieza, filtrado, `groupby` y agregaciones por categoría, y comparar la solución en Pandas contra una implementación con `Dask DataFrame` o `PySpark`. El análisis debe considerar si el costo de coordinar particiones compensa o no el beneficio obtenido.
- **Ejemplo para el Experimento D:** Simular un pipeline donde cada tarea requiera enviar datos, serializarlos, procesarlos y devolver resultados. Puede modelarse el overhead agregando tiempos de preparación, transferencia o unión de resultados para mostrar que aumentar workers o particiones no siempre produce speedup lineal. A partir de ello, justificar qué enfoque recomendarían para un servidor único, para varias fuentes de datos y para una plataforma en cloud.

* Registro mínimo obligatorio

Toda entrega debe registrar explícitamente:

- versión de Python utilizada;
- sistema operativo;
- descripción breve del equipo usado (CPU y memoria aproximada);
- librerías empleadas;
- parámetros de prueba definidos por el grupo;
- tiempos obtenidos;
- interpretación de resultados.

* Advertencia de integridad académica y uso de herramientas de IA

Los ejemplos de código incorporados en esta guía son **solo referenciales**. Su propósito es orientar la forma general del problema, mostrar una posible estructura de trabajo y ayudar a encuadrar el laboratorio. No deben entenderse como una solución completa ni como una plantilla para copiar.

Cada grupo debe construir su propio desarrollo, lo que implica:

- proponer sus propios casos de prueba;
- definir sus propios tamaños de entrada;
- establecer sus propias suposiciones experimentales;
- justificar la elección de parámetros;
- redactar con autonomía sus análisis y conclusiones.

El uso de herramientas de inteligencia artificial no está permitido como sustituto del razonamiento humano del grupo. En particular, se penalizará subir tal cual las instrucciones del laboratorio a cualquier sistema de IA y presentar luego resultados, código, análisis o conclusiones entregadas de manera determinista, genérica o sustancialmente similares a las de otros grupos.

Se espera que, si un grupo utiliza alguna herramienta de apoyo, exista trabajo intelectual propio, adaptación crítica, reformulación de supuestos, selección justificada de ejemplos y análisis genuino. La simple delegación de la tarea a una IA no constituye aprendizaje ni trabajo válido para esta asignatura. No se aceptarán entregas esencialmente idénticas entre grupos, aunque existan cambios superficiales de nombres de variables, comentarios, orden del código o formato del texto. Si se detecta **alta similitud** entre ejercicios de distintos grupos, ya sea por copia directa o por uso acrítico de IA que produzca respuestas comunes, **todo el conjunto de estudiantes involucrados será sancionado con nota 1.0.**

3. Productos Esperados

Los entregables deben evidenciar la ejecución completa de la investigación teórica, los experimentos prácticos y el análisis de resultados. Se espera un conjunto ordenado que permita reproducir los experimentos, validar las conclusiones y justificar decisiones de infraestructura con base técnica.

- Informe técnico en PDF con análisis conceptual, metodología, resultados, visualizaciones y conclusiones.
- Código fuente en Python (.py o .ipynb) que implemente los cuatro experimentos y permita reproducir los resultados.
- Datos de entrada, particiones, resultados tabulares y métricas experimentales en formatos legibles.
- Visualizaciones de tiempos, speedup, eficiencia, escalabilidad y costo de coordinación.
- Documentación del entorno experimental, incluyendo hardware, librerías, configuración local del framework distribuido y fecha de ejecución.
- Repositorio con todos los archivos de entrega y un README con instrucciones claras de ejecución.

3.1 Estructura mínima del informe final

- a. **Portada:** Título, nombre(s), fecha, curso y actividad.
- b. **Resumen Ejecutivo:** Síntesis de los objetivos, metodología, principales hallazgos y recomendación general.
- c. **Marco Teórico:** Procesamiento paralelo vs distribuido, paralelismo de datos y tareas, multicore, GPU, nodos, cluster, tolerancia a fallas, escalamiento vertical y horizontal.
- d. **Metodología Experimental:** Definición del entorno, variables controladas, diseño de experimentos, tamaños de entrada y criterios de comparación.
- e. **Resultados:** Tablas, métricas y gráficos correspondientes a los experimentos A, B, C y D.
- f. **Análisis y Discusión:** Interpretación de resultados, discusión de overhead, memoria compartida, coordinación, contención, escalabilidad y casos de uso adecuados.
- g. **Decisiones Técnicas por Escenario:** Recomendación razonada para los tres casos aplicados solicitados.
- h. **Conclusiones y Recomendaciones:** Hallazgos finales, limitaciones del trabajo y sugerencias de mejora.
- i. **Anexos y Referencias:** Código relevante, datos crudos, comandos de ejecución y fuentes consultadas.

3.2 Estructura mínima del repositorio y entrega

- a. **README:** Instrucciones para instalar dependencias, ejecutar el código y reproducir los experimentos.
- b. **Código Python:** Scripts o notebooks organizados por experimento y documentados.
- c. **Datos:** Archivos de entrada, particiones y resultados de métricas.
- d. **Visualizaciones:** Gráficos comparativos de tiempos, speedup, eficiencia y escalabilidad.
- e. **Entorno experimental:** Registro de hardware, sistema operativo, versiones de librerías, framework distribuido utilizado y fecha de ejecución.
- f. **Informe final:** Documento PDF con la narrativa completa del trabajo.

3.3 Recomendaciones y observaciones adicionales

- a) Mantener una redacción formal, clara y precisa.
- b) Numerar todas las figuras, tablas y anexos, y referenciarlos en el texto.
- c) Revisar ortografía, coherencia y consistencia metodológica antes de la entrega.
- d) Documentar explícitamente los supuestos del experimento, especialmente cuando el framework distribuido se ejecute en modo local.
- e) Reportar con claridad el número de workers, particiones, tamaños de entrada, tiempos medidos y criterio usado para calcular speedup y eficiencia.
- f) Diferenciar con evidencia cuándo el cuello de botella proviene de CPU, memoria, I/O o coordinación.
- g) Citar las fuentes consultadas y declarar el uso de IA o código ajeno cuando corresponda.
- h) La copia de trabajo ajeno será considerada plagio y sancionada según normativa institucional.

4. Rúbrica de Evaluación

La evaluación considera la profundidad conceptual, la calidad de la implementación en Python, el rigor experimental, la calidad del análisis y la presentación profesional del trabajo.

Escala de evaluación:

- 90–100: Excelente.
- 80–89: Muy bueno.
- 70–79: Bueno.
- 60–69: Suficiente.
- Menos de 60: Insuficiente.

Criterio	Excelente (4)	Bueno (3)	Suficiente (2)	Insuficiente (1)
A. Investigación teórica (30 %)				
Conceptos de paralelismo y distribución	Preciso, completo y con comparación crítica.	Correcto y bien explicado.	Parcial con omisiones relevantes.	Incompleto o incorrecto.
Criterios de decisión y casos de uso	Relaciona teoría, límites y escenarios aplicados.	Relaciona conceptos con ejemplos adecuados.	Ejemplos superficiales o poco justificados.	No demuestra comprensión aplicada.
B. Implementación y reproducibilidad (25 %)				
Código funcional y organizado	Implementación robusta, clara y bien documentada.	Funcional con buena organización.	Funcional parcial o con errores menores.	No funcional o desordenada.
Uso de herramientas paralelas/distribuidas	Usa correctamente librerías y configura bien los experimentos.	Buen uso con detalles menores por mejorar.	Uso básico o parcialmente correcto.	Uso incorrecto o ausente.
C. Diseño experimental (15 %)				
Variedad de configuraciones	Cubre todos los experimentos y configuraciones requeridas.	Cubre la mayoría con buen nivel de detalle.	Cobertura parcial de configuraciones.	Configuración insuficiente.
Control de variables y metodología	Metodología rigurosa y consistente.	Metodología adecuada.	Metodología básica.	Metodología deficiente.
D. Análisis de resultados (20 %)				
Interpretación de speedup y overhead	Análisis profundo, cuantitativo y bien fundamentado.	Buen análisis con evidencia suficiente.	Análisis limitado o superficial.	Análisis pobre o ausente.
Recomendación técnica por escenarios	Conclusiones claras, aplicables y bien justificadas.	Conclusiones válidas con justificación aceptable.	Recomendaciones generales poco precisas.	Sin recomendaciones útiles.
E. Presentación y formato (10 %)				
Claridad y estructura del informe	Informe ordenado, profesional y coherente.	Informe adecuado con detalles menores.	Informe poco ordenado.	Informe desordenado o difícil de leer.
Visualizaciones y documentación	Gráficos claros, bien integrados y documentación completa.	Gráficos adecuados y documentación suficiente.	Gráficos limitados o documentación incompleta.	Sin visualizaciones relevantes o sin documentación.

Cuadro 1: Rúbrica de evaluación detallada - Actividad de paralelismo local y procesamiento distribuido

Referencias Bibliográficas

1. Python Software Foundation. *concurrent.futures — Launching parallel tasks*. Recuperado de <https://docs.python.org/3/library/concurrent.futures.html>
2. Python Software Foundation. *multiprocessing — Process-based parallelism*. Recuperado de <https://docs.python.org/3/library/multiprocessing.html>
3. NumPy Developers. *NumPy documentation*. Recuperado de <https://numpy.org/doc/>

4. Dask. *Dask documentation*. Recuperado de <https://docs.dask.org>
5. Apache Spark. *Apache Spark documentation*. Recuperado de <https://spark.apache.org/docs/>
6. NVIDIA. *CUDA Zone*. Recuperado de <https://developer.nvidia.com/cuda-zone>
7. Hwang, K., Dongarra, J., & Fox, G. (2013). *Distributed and Cloud Computing: From Parallel Processing to the Internet of Things*. Morgan Kaufmann.
8. Tanenbaum, A. S., & Steen, M. van. (2017). *Distributed Systems* (3rd ed.). Pearson.
9. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
10. Fuentes oficiales adicionales, notas de clase y documentación técnica citadas explícitamente en el informe final.